

Expanding Azure Logic Apps Capabilities to internal Resources

 minder-security.ghost.io/expanding-azure-logic-apps-capabilities-to-internal-resources

Richard Minder

April 14, 2025



The goal of using On-Premises Gateways is to enable Azure Logic Apps and Sentinel Playbooks to work with on-premises resources. Many organizations implement security products **inside** their private networks, without wanting to expose them to the internet.

SOCs all over the world implement local, on-premises SOAR solutions such as Splunk SOAR (on-premises) for these reasons, while Microsoft also offers a strong set of tools.

Using On-Premises Data Gateways, I aim at showcasing that Azure Logic Apps' capabilities go beyond public-facing infrastructure and cloud-based solutions.

Links:

- [Install an on-premises data gateway | Microsoft Learn](#)
- [Connect to on-premises data sources - Azure Logic Apps | Microsoft Learn](#)
- [The Ultimate Guide to Connecting to On-Premise API's using Azure Logic Apps | by Navaneeth Sen | Medium](#)

What are Data Gateways

One of the most important considerations when connecting Azure Logic Apps to internal resources is security.

- On-Premises Data Gateways are installed on a host within the local network
- The host sends requests **TO** the Azure Cloud, meaning that the host must not be publicly accessible to work (NAT)
- Through a local AD Join, access can be monitored, restricted and adjusted as wished

Basically, an On-Premises Data Gateway is an application you install on an internal windows host, which will send requests to the azure cloud, receive instructions from Logic Apps, and then forward them to internal resources.

This setup enables cloud services to send requests to the gateway, which forwards those instructions to the designated internal resources. It ensures that your data remains on-premises while still leveraging the power of Azure cloud services.

Requirements

To run On-Premises Data Gateways, the following prerequisites must be met:

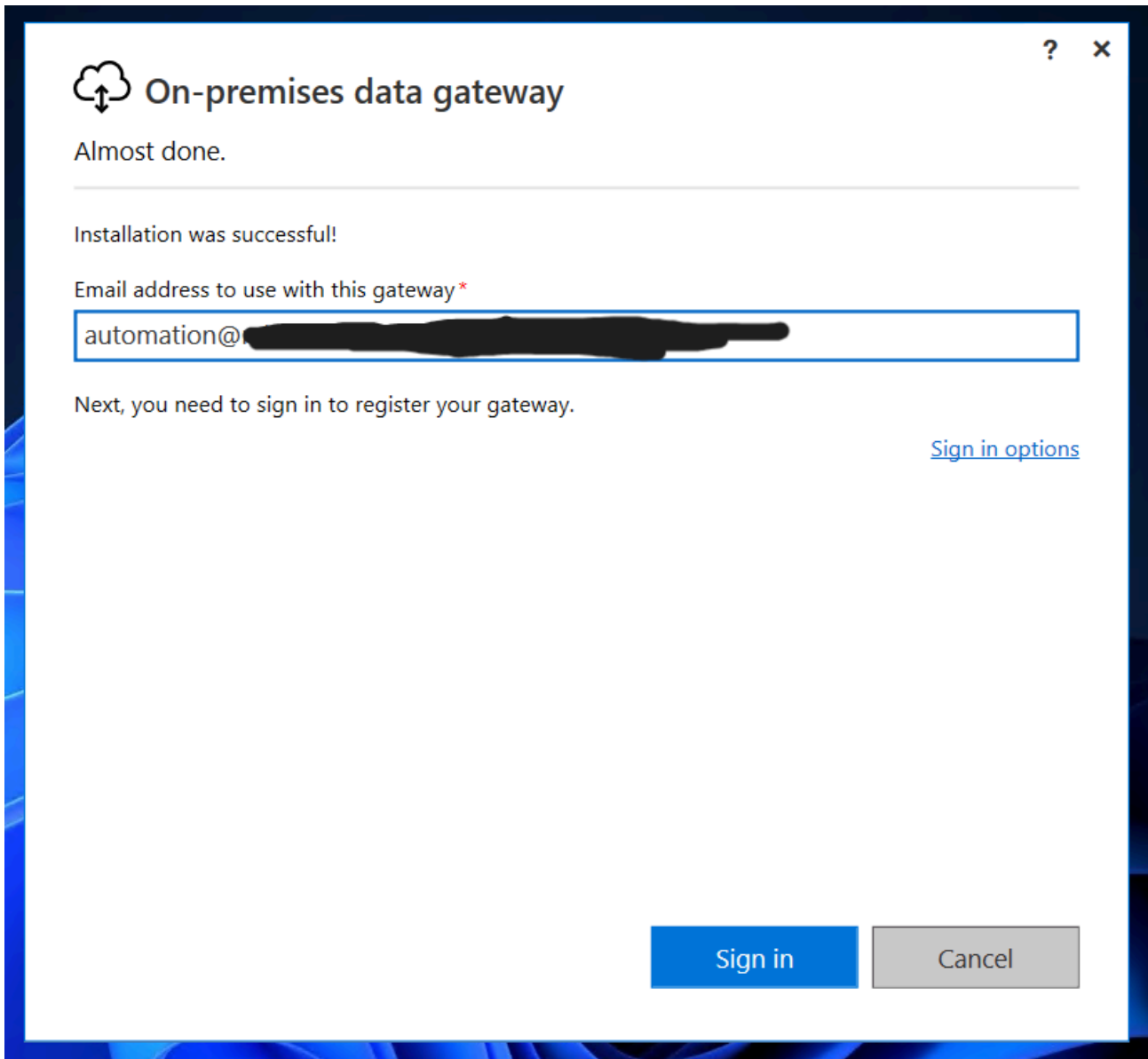
- On-Premises Host running On-Premises Gateway
 - Windows Host within the Network
 - .NET 4.8
 - Domain joined if resources within AD should be accessed
 - For Server instances → Core is not supported
- A user within the Entra ID Tenant with “Gateway Administrator” Role
 - This role could not be found, Contributor was used instead (not recommended for prod implementations)

Test Lab Setup

1. Create Hyper-V VM (template: Windows 11 dev environment)

2. Enable nested virtualization using `Set-VMProcessor -VMName <VMName> -ExposeVirtualizationExtensions $true`
3. Install python on WSL
4. Download and run the Gateway Installer (see below)

[Download On-premises data gateway from Official Microsoft Download Center](#)
[Keep your data in Power BI, PowerApps, Logic Apps, and Power Automate up to date using the on-premises data gateway.](#)
[Microsoft Store - Download Center](#)



 **On-premises data gateway** ? x

Almost done.

Installation was successful!

Email address to use with this gateway *

automation@[REDACTED]

Next, you need to sign in to register your gateway.

[Sign in options](#)

Sign in Cancel



On-premises data gateway



You are signed in as automation@mindersecurity.onmicrosoft.com and are ready to register the gateway.

- ☒ Register a new gateway on this computer.
- ☐ Migrate, restore, or takeover an existing gateway.
 - Move a gateway to a new computer
 - Recover a damaged gateway
 - Take ownership of a gatewayThe old gateway will be disconnected.

Next

Cancel

Microsoft Store



On-premises data gateway



You are signed in as automation@mindersecurity.onmicrosoft.com and are ready to register the gateway.

New on-premises data gateway name *

gateway-test

☐ Add to an existing gateway cluster [Learn more](#)

Recovery key (8 character minimum) *

.....

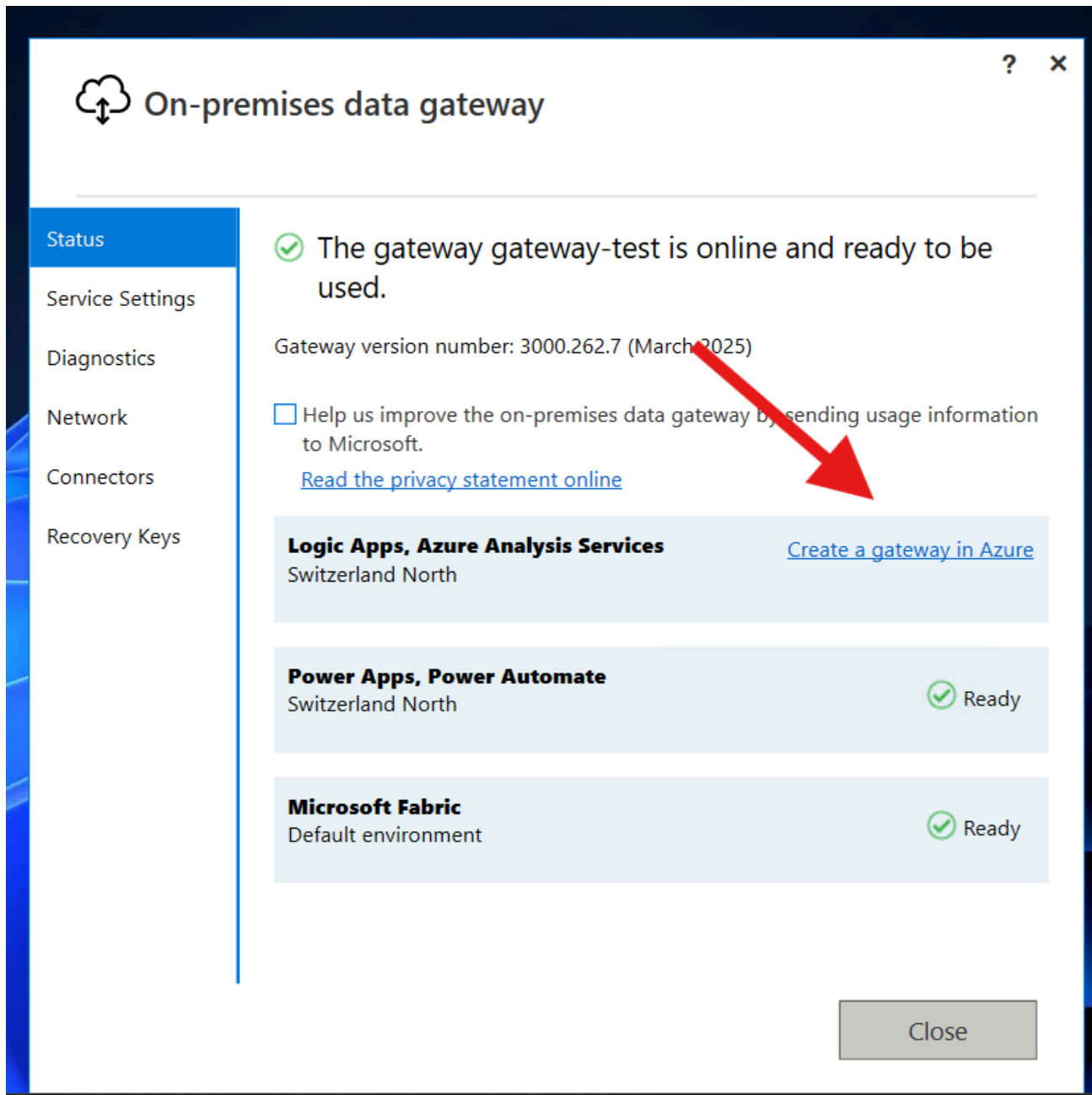
This key is needed to restore the gateway and can't be changed. Record it in a safe place.

Confirm recovery key *

.....

We'll use this region to connect the gateway to cloud services: Switzerland North [Change Region](#)

[Provide relay details \(optional\)](#) By default, Azure Relays are automatically provisioned



You will then be forwarded to the "Create Gateway" blade within Azure Portal.

Make sure you authenticate to the Azure Portal using **the same account** as you authenticated with to the on-premises app (above)

Create a gateway ...

Power BI, Microsoft Flow, Logic Apps, and PowerApps. [Learn more](#)

Project details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription * ⓘ

Resource group * ⓘ

[Create new](#)

Instance details

Name *

Region * ⓘ

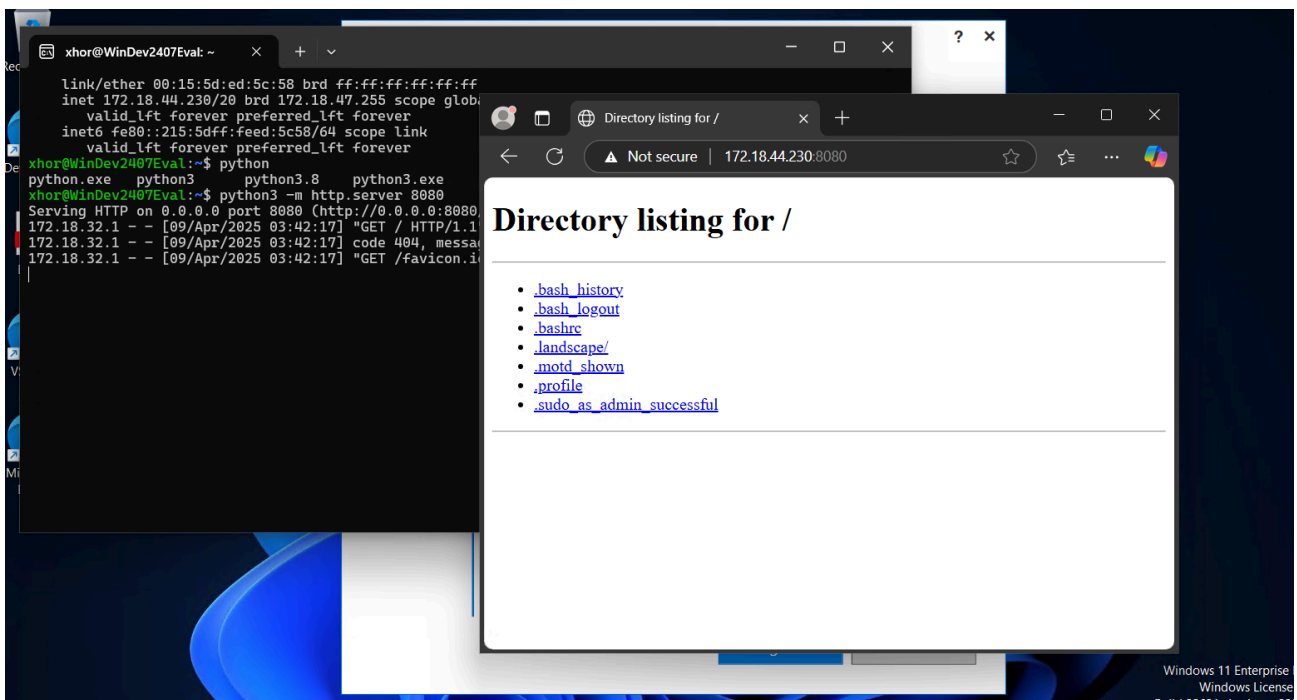
Installation Name *

Select from available installed gateways. For details click here.

[Logic apps gateway connection](#)

[Previous](#) [Next](#) [Review + create](#)

You can then go ahead and test the implementation by running a local HTTP webserver via `python3 -m http.server 8080`, simulating an API we wish to access:



Then create a Logic App to interact with it:

- You can choose the Trigger as you wish

- The connector used to interact with On-Premises resources **must** be “HTTP with Entra ID” in order to be able to work with the gateway

The screenshot shows the Logic app designer interface for a logic app named 'la-test'. The left sidebar contains navigation options like Overview, Activity log, Access control (IAM), Tags, Diagnose and solve problems, Resource visualizer, and Development Tools. The main canvas displays a workflow with two steps: 'When a HTTP request is received' followed by 'Invoke an HTTP request'. The right-hand pane is titled 'Create connection' and shows the configuration for the 'Invoke an HTTP request' connector. The 'Connect via on-premises data gateway' checkbox is checked. The configuration fields include:

- Connection Name: gateway-test-azure
- Base Resource URL: http://172.18.44.230:8080/
- Microsoft Entra ID Resource URI (Application ID URI): A text area with a placeholder explaining the identifier.
- Username: Username credential
- Password: Password credential
- Authentication Type: Anonymous
- Gateway: Subscription (Azure subscription 1) and Gateway (gateway-test).

 A 'Create new' button is at the bottom right of the configuration pane.

You can then validate that the request was successful by going to **Run history** and selecting your last run:

The screenshot shows the Logic app designer interface with the 'Run history' tab selected. The left sidebar is the same as in the previous image. The main canvas shows the workflow with 'When a HTTP request is received' and 'Invoke an HTTP request' steps, both marked with green checkmarks and durations (0s and 0.8s respectively). The right-hand pane displays the details of the selected run. The 'Parameters' tab is active, showing the 'Body' of the request as an HTML document listing directory contents. The 'PROPERTIES' section at the bottom shows the 'Start time' and 'End time' of the run, both as '4/9/2025, 12:47:12 PM (Local time)'.

Where to go from here

Enable playbooks to perform actions on your on-premises infrastructure leverages your possibilities regarding automation, while I focus on:

- **Sentinel as a SOAR:** While usually implemented as a SIEM/SOAR combination, Data Gateways enable SOC's to use Sentinel as a standalone SOAR through forwarding alerts from on-premises sources.
- **Automated Investigation:** Having the ability to store data on-premise and query it using Logic Apps enables security professionals to acquire more evidence during investigations.
- **Automated Response:** With Logic Apps/Playbooks being able to perform actions on on-premises infrastructure, more response possibilities arise as local security tooling can be utilized.

Thank you for reading my blog post!